

prog2 lab 2025

Debugging and timing

GDB

- gdb e' il debugger usato per programmi C e C++
tutorial on line:
- <https://developers.redhat.com/blog/2021/04/30/the-gdb-developers-gnu-debugger-tutorial-part-1-getting-started-with-the-debugger>

GDB

- gdb permette di eseguire in modo controllato il vostro programma
- permette anche, se il vostro programma va in crash, di esaminarne lo stato esattamente un istante prima
- gdb non ha GUI, funziona a linea di comando testuale
- e' una scomodita' ma cosi' si puo' usare anche su dispositivi piu' elementari
- Il vero C programmer non si fa scoraggiare da questi disagi..

GDB

- file .gdbinit nella home directory
- help <comando>
- alias gdb='gdb -q'

su LINUX, per avere un core dump:

#pura magia.....

- sudo systemctl disable apport
- sudo sysctl -w kernel.core_pattern=core.%u.%p.%t
- ulimit -c unlimited

GDB

- Per usare il debugger, occorre arricchire l'eseguibile di informazioni sul codice, ad esempio i nomi dei simboli del sorgente.
- Questo si fa aggiungendo l'opzione '-g'
- gcc -g <lista sorgenti> -o <nome eseguibile>
- notare che l'opzione -g e' incompatibile di solito con '-O' , che serve per ottimizzare il codice
- ho omesso -Wall come opzione perche' non serve a gdb , ma dovete sempre metterla!!

GDB

- a questo punto , basta eseguire:
- `gdb <nome eseguibile>`

il programma ha tantissimi comandi, ma a noi bastano alcuni di partenza, il resto lo potete studiare all'occorrenza

GDB i comandi

b(reak) <nome funzione> oppure <nome file>:<numero linea>

-> ferma il programma se passa in quel punto

n(ext)

-> esegue lo statement successivo del sorgente

s(tep)

-> come next, ma se lo statement e' una function call,
entra dentro alla funzione

c(ontinue)

-> prosegui fino al prossimo breakpoint

GDB i comandi

l(ist)

-> stampa il sorgente attorno al punto di esecuzione

p(rint) <nome simbolo>

-> stampa il valore della variabile

info locals

-> stampa tutte le variabili locali

where

-> stampa lo stack delle chiamate

GDB

- esempio di breakpoint condizionato:
break main.c:36 if i > 2
- osservare quando una variabile viene modificata:
watch i

.....

In generale

- Il debugger e' uno strumento utilissimo
- ma in molti la cosa piu' efficiente e' basarsi sul testing
- ad ogni modifica, se possibile, rieseguire tutta la test suite a disposizione,
- se le modifiche sono piccole ci si accorge immediatamente di un errore magari banale.
- utilizzare il debugger comporta comunque un certo dispendio di tempo
- nella pratica professionale tutti gli ambienti di sviluppo dispongono di comodi debugger con GUI (magari basati su gdb)

la printf()

- in generale e' bene creare una o piu'funzioni di stampa a terminale tramite printf()
- utile per visualizzare la struttura dei dati
- createla subito, anche se siete all'esame, e' una funzione rapida da creare che vi fara' risparmiare un sacco di tempo

Esercizio gdb

- troviamo l' errore nel sorgente buggyprime.c (prestato dal corso ufficiale) e semplificato

Tempo di esecuzione

- Il C si usa per la sua velocità e quindi è utile verificare i tempi di esecuzione ad esempio di 2 versioni differenti di uno stesso programma
- Esistono strumenti molto sofisticati per fare questo ma esulano dal questo corso.
- È facile tuttavia costruire un piccolo modulo che rileva la tempistica grezza.
- esaminiamo la cartella sorgenti di Lab0

uso di tempi.c

se si invoca `miotempo(NULL);` // il tempo e' aggiornato

se si invoca `miotempo("ciao");` // stampa la differenza di tempo dall'ultima chiamata

`cc -l. -O main.c tempi.c ; a.out`

ricordatevi di testare con `-O` ; due programmi di velocita' simile normalmente, possono divergere parecchio se ottimizzati

main.c

- ci sono esempi di calcolo dei tempi di esecuzione dello stesso loop con tre indicizzazioni diverse.
- come vedete i tempi sono equivalenti, scegliete se potete la notazione piu' semplice
- interessante notare che il primo loop, qualunque sia dura parecchio piu' a lungo, come mai?

main.c

- il primo loop include il tempo di caricamento in memoria della struttura dati, che e' in questo caso e' lungo
- attenzione quindi a confrontare i tempi con criterio !
- le tre versioni dovrebbero sulla carta produrre tempi identici..